Day 3

Tools, Function Calling & the Action Space

Lecturer: Haoyuan Jiang, Xuerui Zhang, Zhenyu Wang

Learning objectives

- **Explain the ReAct feedback loop.**
- **Distinguish context, trace, observation space, and action space.**
- **Select tools based on task needs and capabilities.**
- **Apply basic tool-design and discovery principles.**
- **Explain how Skills, events, user communication, and plugins extend agents.**

Content

01 ReAct architecture

02 Context, observation and action space

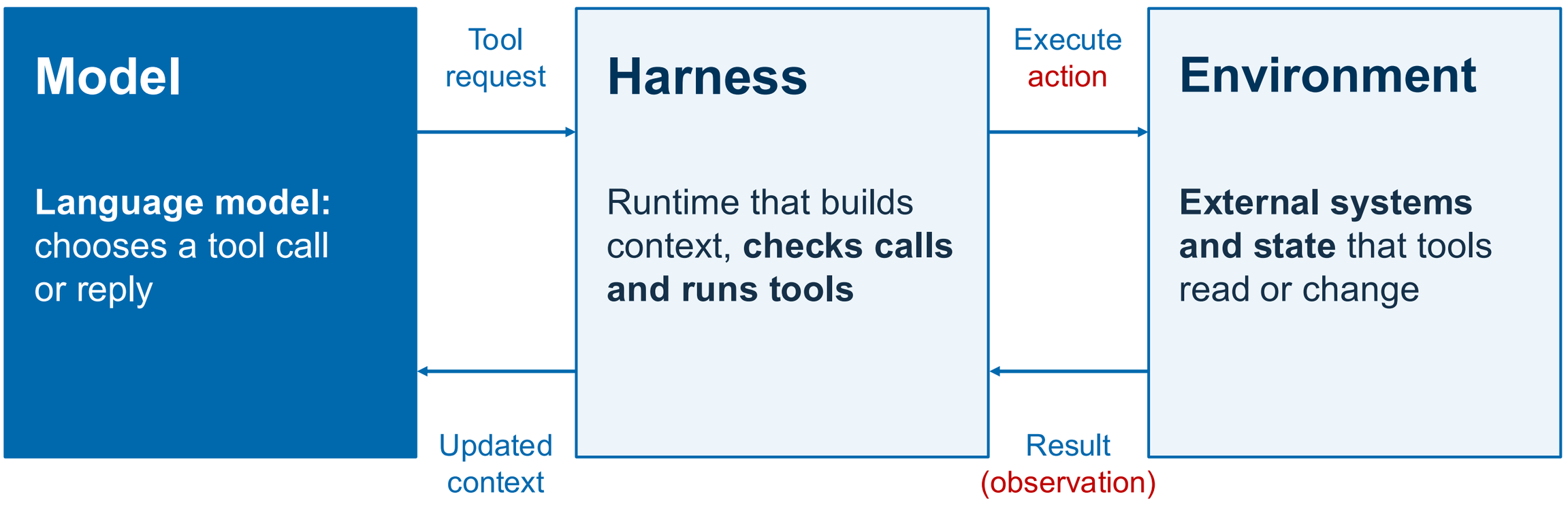
03 Tools capabilities and selection

04 Tools design and discovery

05 Skills, events and Plugins

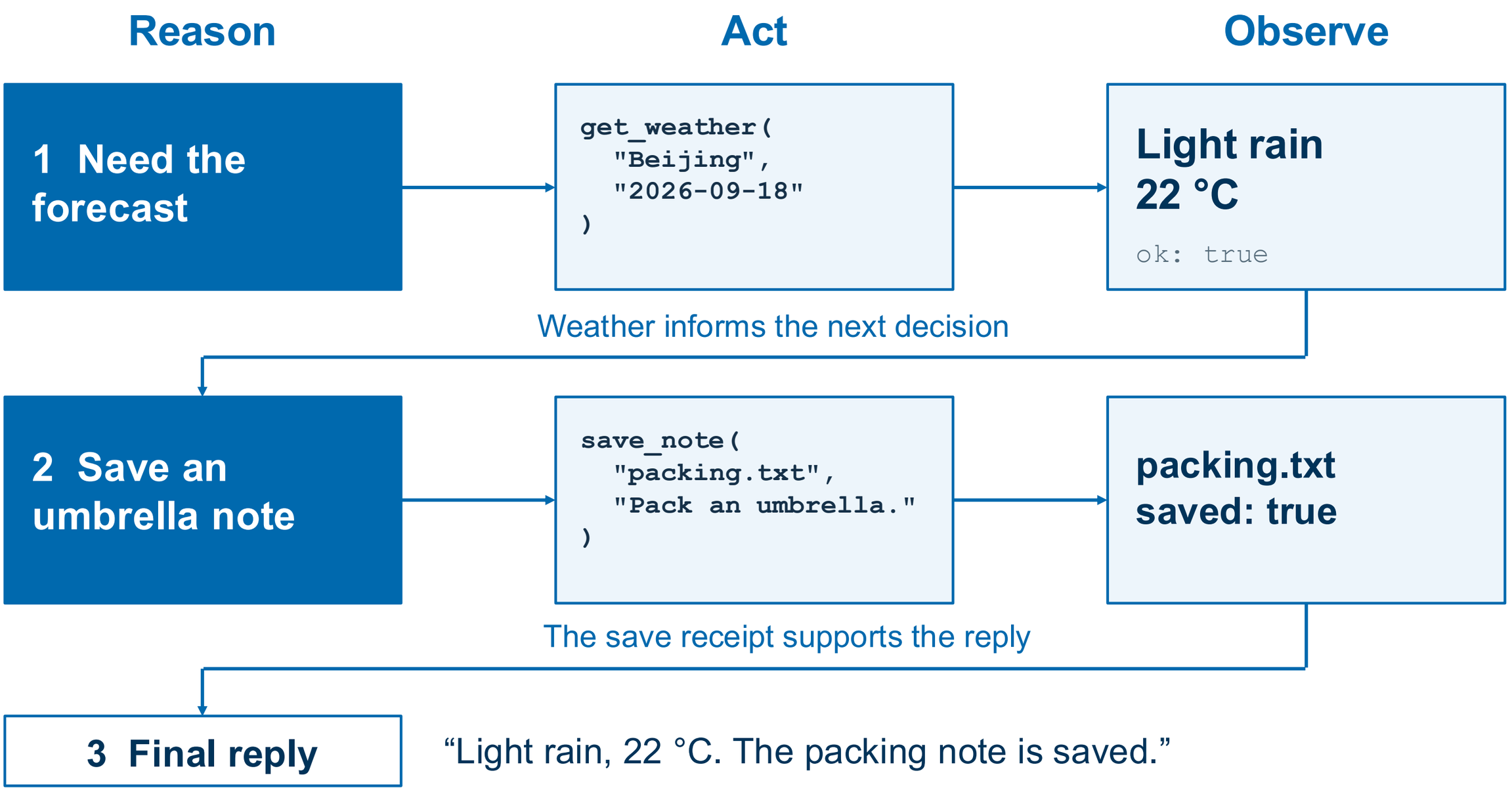
0 Model, harness and environment

- **Harness** determines how the Agent thinks and acts (software layer).
  - Harness (origin): a set of straps and fittings by which a horse is controlled by its driver.
- **Runtime** determines where it runs and under what constraints (infrastructure layer).
  - The runtime connects model decisions to the outside world.



1 ReAct Example: a weather-based packing note

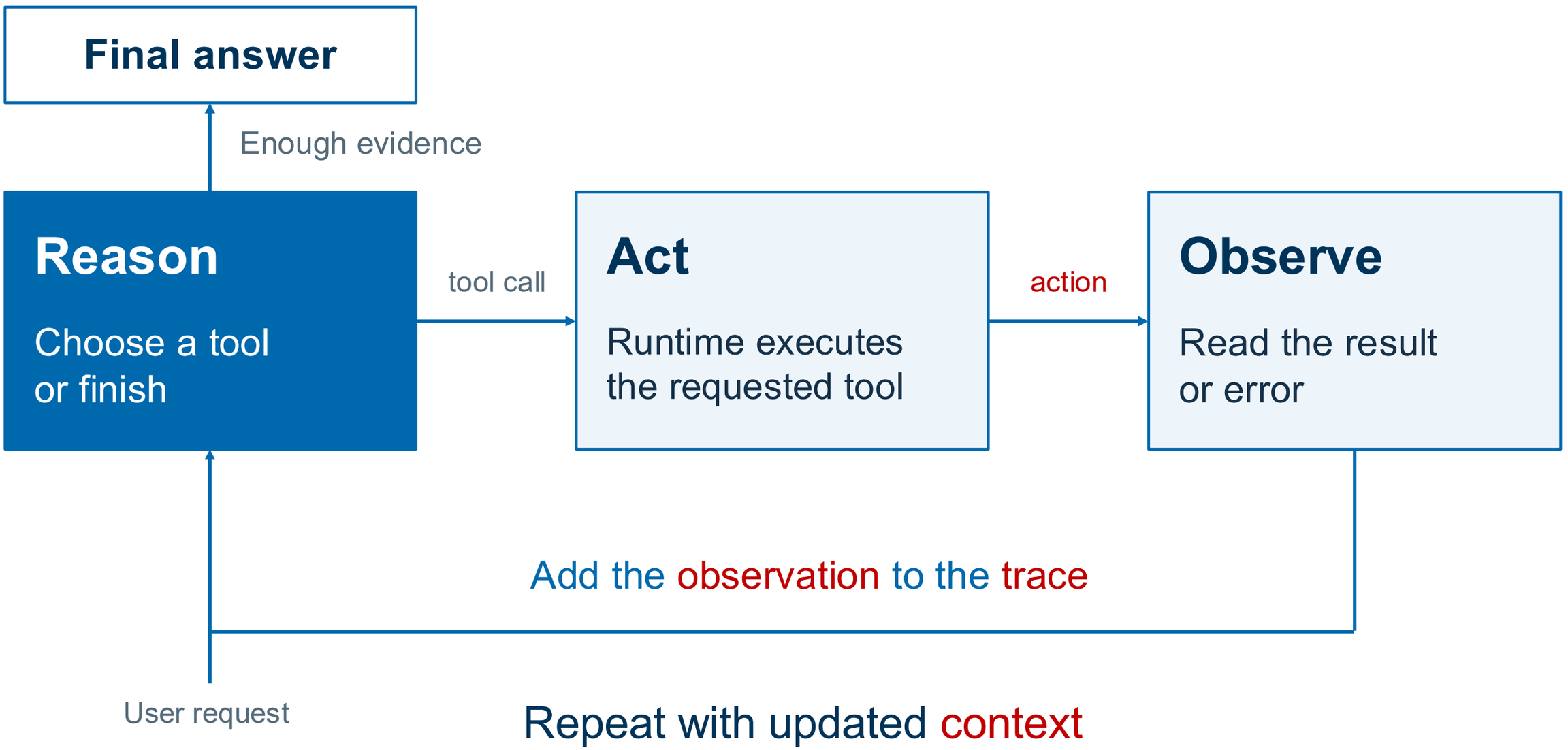
User request: check Beijing weather in 2026-09-18 and save a packing note.



“Light rain, 22 °C. The packing note is saved.”

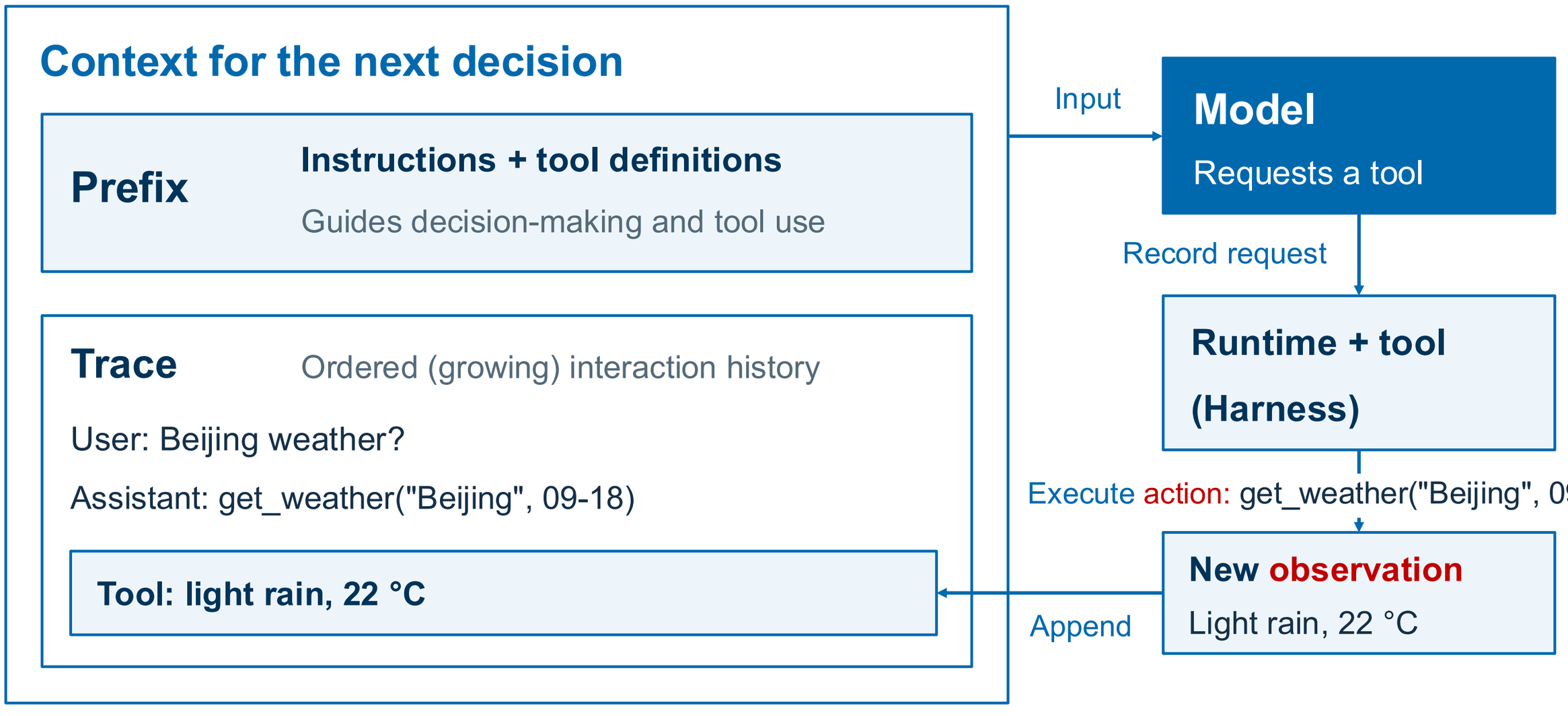
1 The ReAct Feedback Loop

- **REasoning and ACTing** form a loop, the new observations guide the next decision.
- ReAct framework was proposed by Shunyu Yao in “ReAct: Synergizing Reasoning and Acting in Language Models”, 2022.



2 Context: prefix + trace

- **Context**: the model's input for the current decision.
- **Trace**: The (historical) ordered record of user messages, assistant messages and tool results.
- **Action space**: The set of operations the agent can perform, constrained by available tools, valid arguments.
- **Observation space**: The set of possible observations the agent can receive through available interfaces.



Key distinction: Calling a tool is an **action**. Its returned result is an **observation**, which can be appended to the next **context**.

3 Tools

What operation does the tool enable?

Perception
Read information from the environment. → get_weather → retrieve a forecast

Execution
Change files, records or external state. → save_note → write travel advice

Collaboration
Delegate work to another agent. → delegate_task → given docs, write summary, make slide, edit video

Event triggering
Register a future condition or schedule. → register_timer → "notify me at 07:00 tomorrow"
(A timer registration happens now; the event arrives later)

User communication
Ask for input or deliver a notification. → notify_user → send a rain alert

3 Tool definitions – Schema example

Schema constrains arguments; the runtime checks access and executes the function.

Perception: get_weather

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description":
      "Forecast for a city/date.",
    "strict": true,
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string"},
        "date": {
          "type": "string",
          "description": "YYYY-MM-DD"
        }
      },
      "required": ["city", "date"],
      "additionalProperties": false
    }
  }
}
```

Execution: save_note

```
{
  "type": "function",
  "function": {
    "name": "save_note",
    "description":
      "Write a note to a local file.",
    "strict": true,
    "parameters": {
      "type": "object",
      "properties": {
        "path": {"type": "string"},
        "text": {"type": "string"}
      },
      "required": ["path", "text"],
      "additionalProperties": false
    }
  }
}
```

3 Tool selection

Choosing an available tool for the current information gap or required operation.

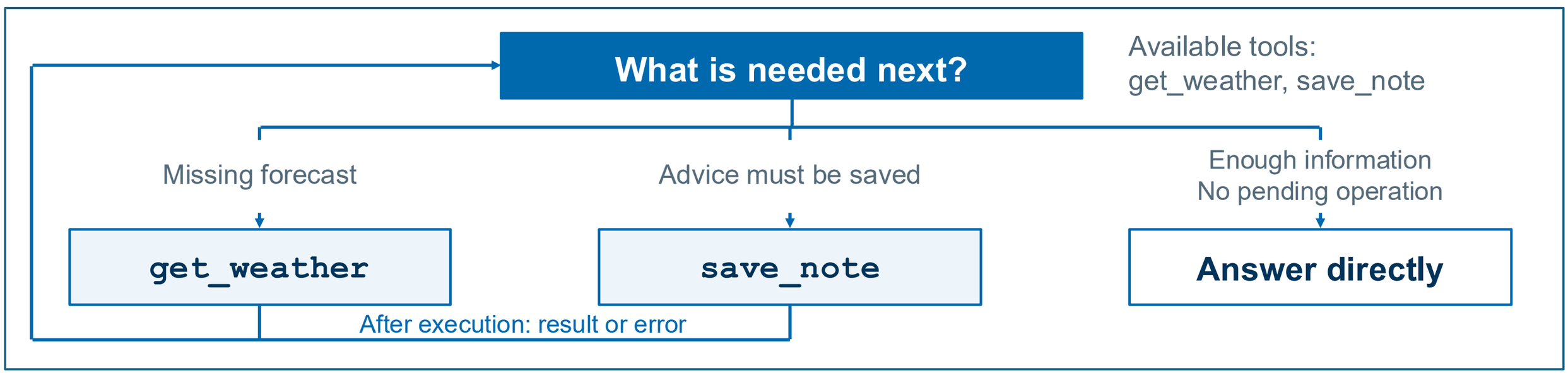
Example: Beijing forecast is missing (model response message)

User request: check Beijing weather in 09-18 and save a packing note.

```
{
  "role": "assistant", "content": null,
  "tool_calls": [{
    "id": "call_weather_1", "type": "function",
    "function": { "name": "get_weather",
      "arguments":
        "{\"city\":\"Beijing\",\"date\":\"2026-09-18\"}"
    }
  }]
}
```

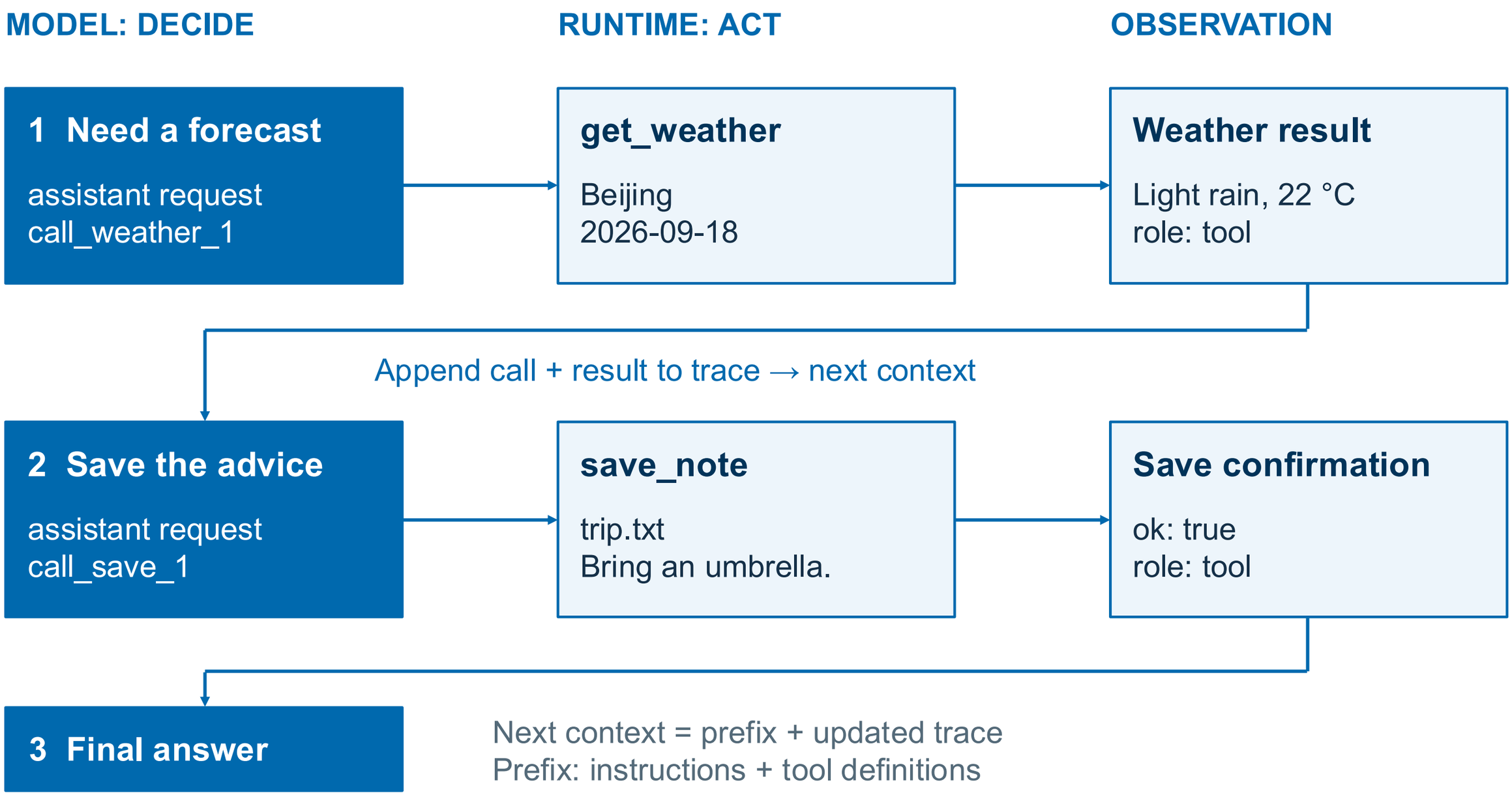
Selection checks

- **Task fit:** weather data.
- **Inputs:** city + date.
- **Cost:** proportionate.
- **Key point:** Use the narrowest sufficient capability, then reassess after the result.



3 Example: ReAct tool-calling loop

User request: retrieve Beijing's forecast in 2026-09-18, then save practical advice.



3 Message format

- **role:** Identifies the message source.
  - **system:** Application instructions for the model.
  - **user:** The task and information supplied by the user.
  - **assistant:** A model-generated reply or tool-call request.
  - **tool:** A result supplied by the runtime for a tool call.
- **content**: Carries the message body.
- **messages**: An ordered array of messages.
- **tools**: All available tool definitions. A request-level array alongside messages.
  - **It is not a message role.**

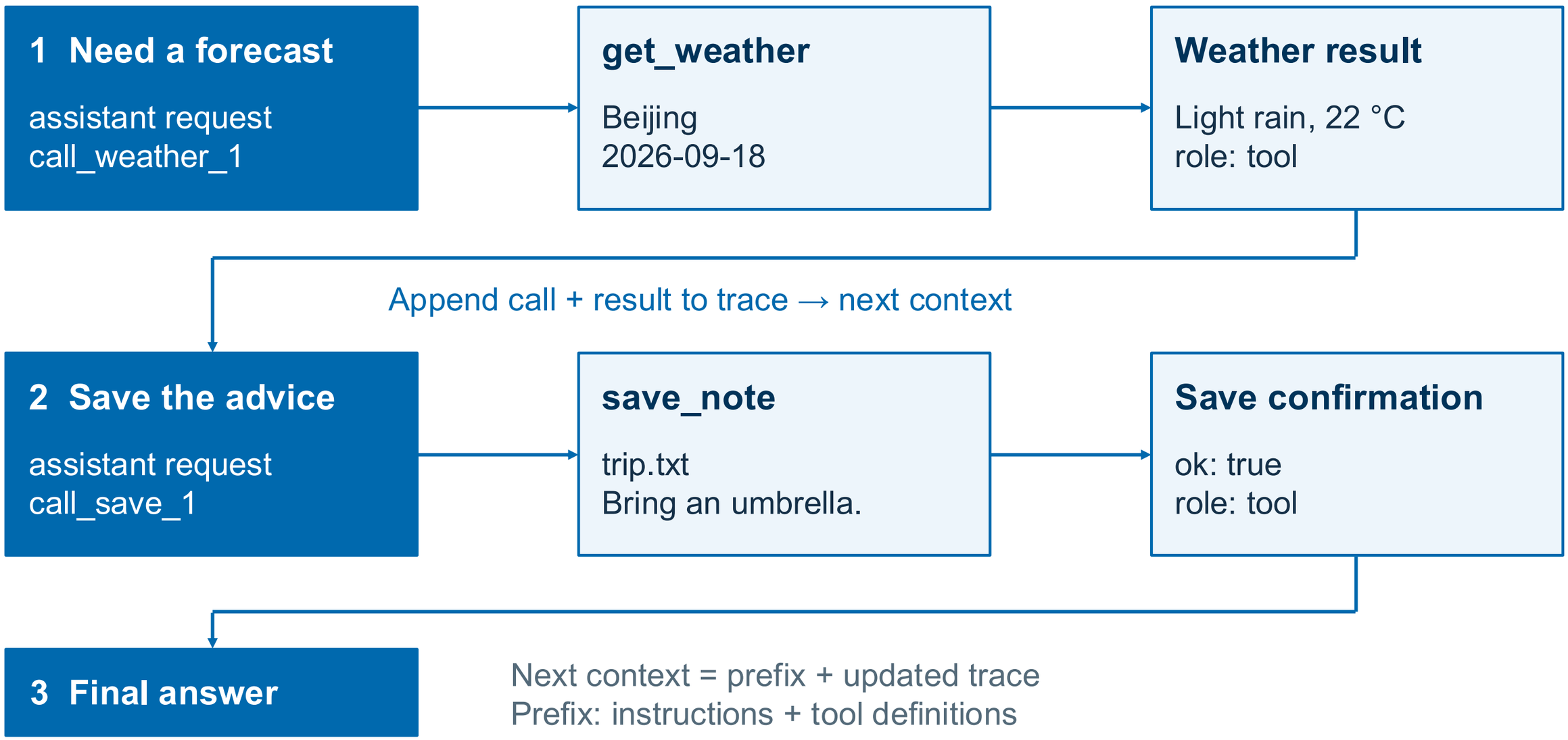
Initial messages (Demo example)

User request: retrieve Beijing's forecast, then save practical advice.

```
[
  {
    "role": "system",
    "content":
      "Use tools;
       report success only after
       confirmation."
  },
  {
    "role": "user",
    "content":
      "Beijing weather on 2026-09-18?
       Save advice to trip.txt."
  }
]
```

3 Example: ReAct tool-calling loop

User request: retrieve Beijing's forecast in 2026-09-18, then save practical advice.



3 ReAct round 1: retrieve weather

Decision: the forecast is missing. Request: get_weather.

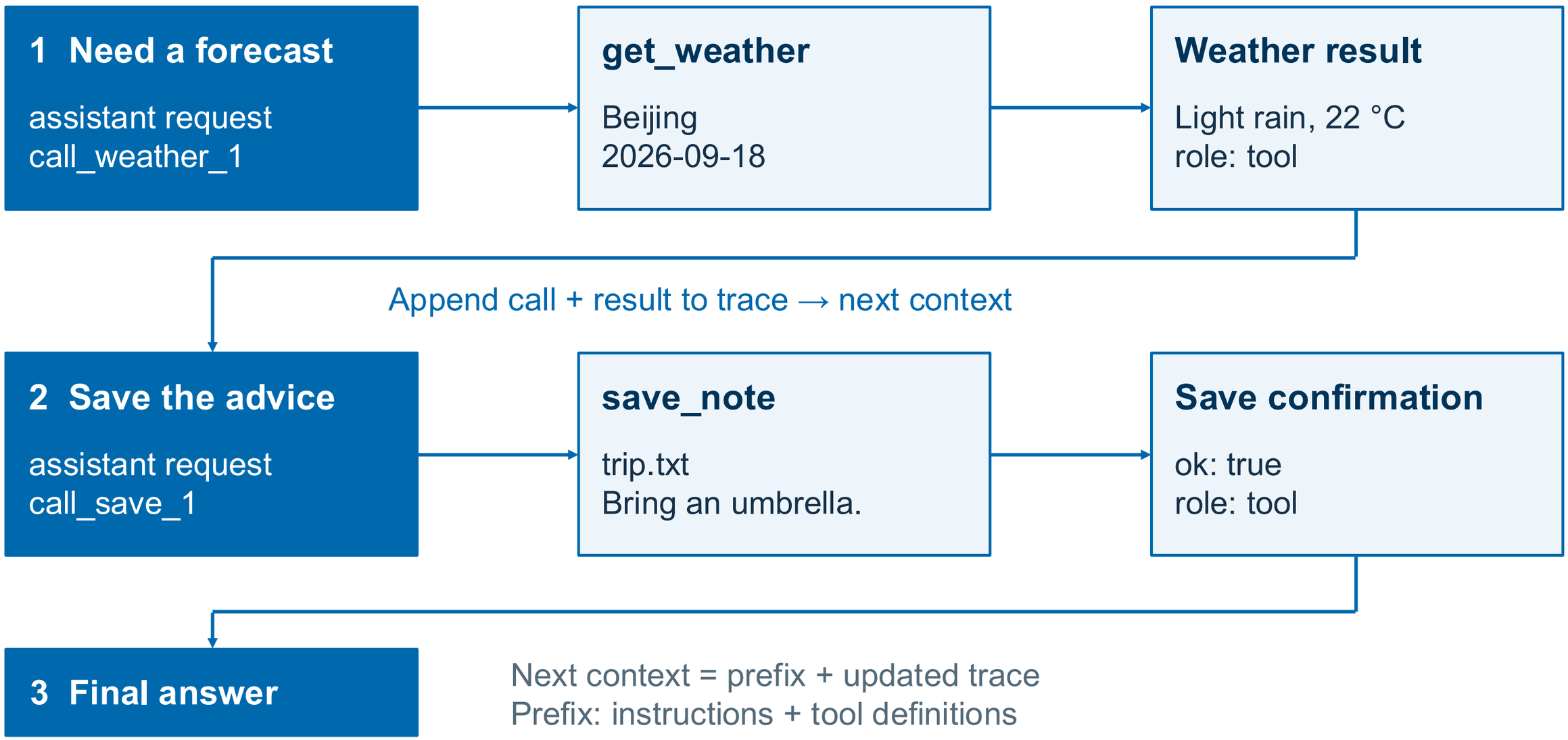
arguments is a JSON string; tool_call_id links each result to its call.


```
[
  {
    "role": "assistant", "content": null,
    "tool_calls": [{
      "id": "call_weather_1",
      "type": "function",
      "function": {
        "name": "get_weather",
        "arguments":
          "{\"city\":\"Beijing\",\"date\":\"2026-09-18\"}"
      }
    }]
  },
  {
    "role": "tool",
    "tool_call_id": "call_weather_1",
    "content": "{\"condition\":\"Light rain\",\"temperature_c\":22}"
  }
]
```


- **Runtime:** Executes the call; appends the returned observation.
- **Next request:** Keep all 4 messages and both tool definitions.

3 Example: ReAct tool-calling loop

User request: retrieve Beijing's forecast in 2026-09-18, then save practical advice.



3 ReAct round 2: save advice

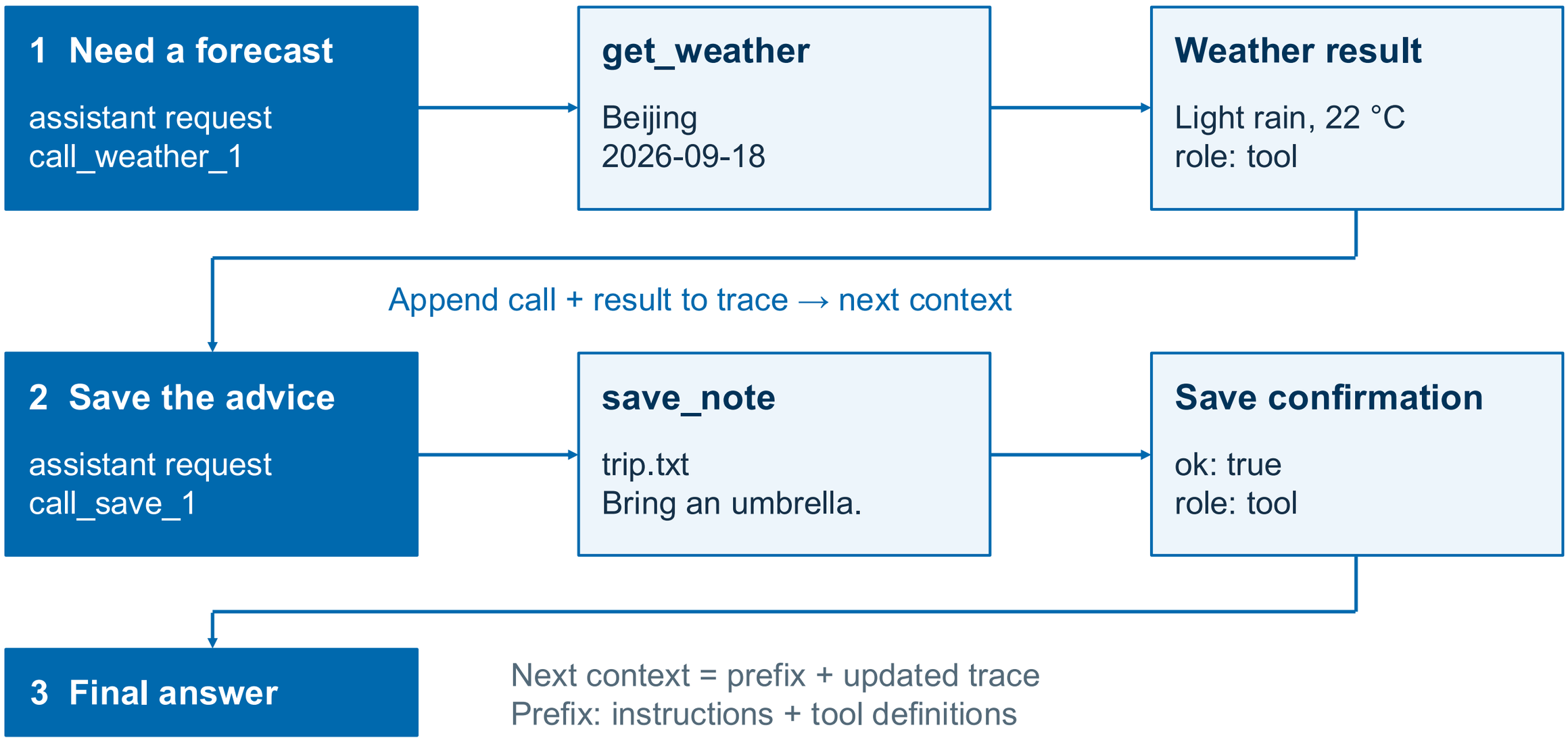
Decision: the forecast is known; saving the advice is still pending.


```
[
  {
    "role": "assistant", "content": null,
    "tool_calls": [{
      "id": "call_save_1",
      "type": "function",
      "function": {
        "name": "save_note",
        "arguments":
          "{\"path\":\"trip.txt\",\"text\":\"Light rain, 22 C. Bring an umbrella.\"}"
      }
    }]
  },
  {
    "role": "tool",
    "tool_call_id": "call_save_1",
    "content": "{\"ok\":true,\"path\":\"trip.txt\"}"
  }
]
```


- **Runtime:** Writes trip.txt; returns a save confirmation.
- **Next request:** Keep all 6 messages and both tool definitions.

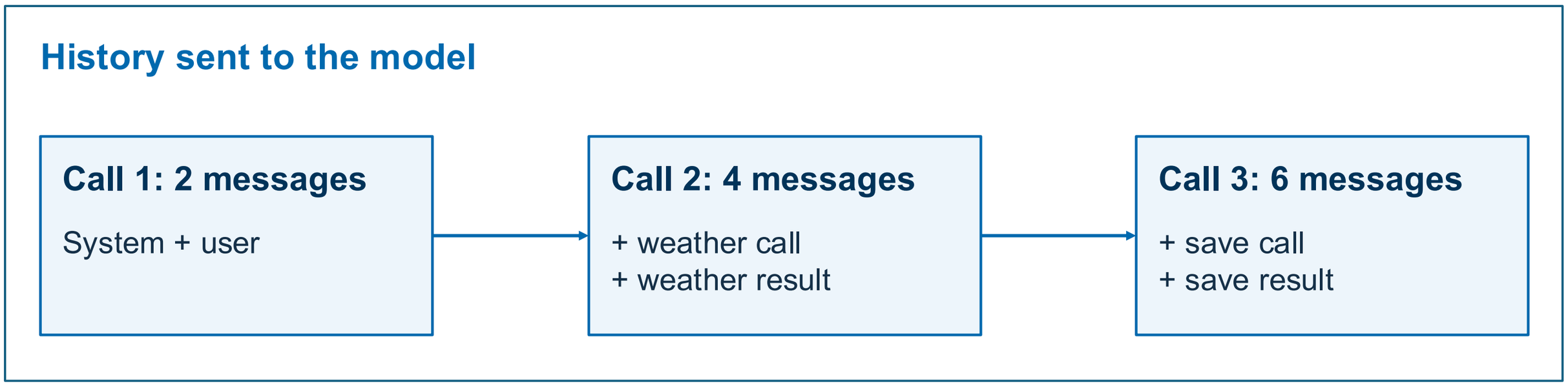
3 Example: ReAct tool-calling loop

User request: retrieve Beijing's forecast in 2026-09-18, then save practical advice.



3 Final reply and accumulated history

Model call 3 uses both observations to finish the task.



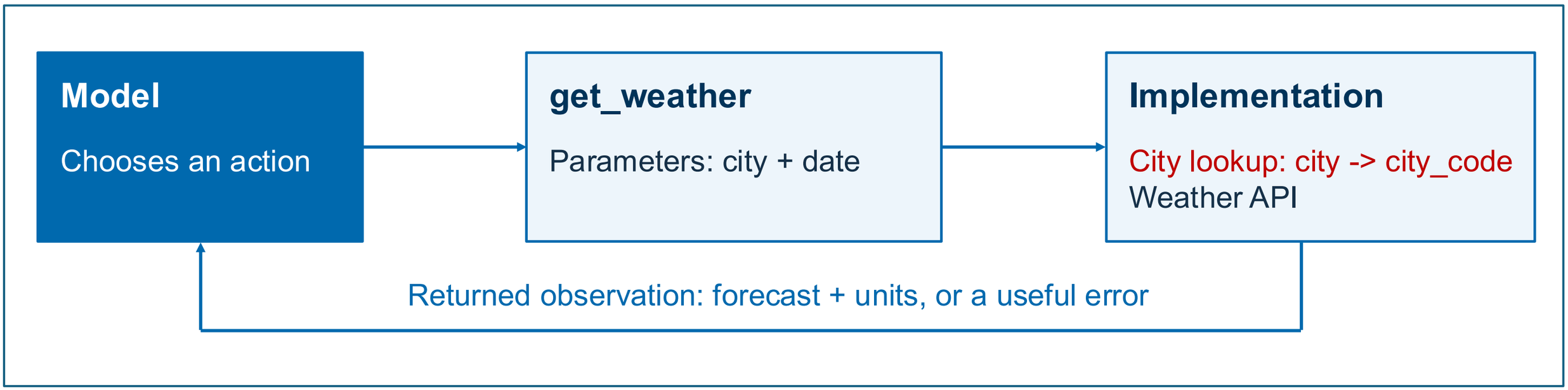

```
{
  "role": "assistant",
  "content":
    "Beijing: light rain, 22 C. Bring an umbrella. Advice saved to trip.txt."
}
```

The final assistant reply becomes message 7. No tool call is needed.

4 Tool design – ACI: Agent–Computer Interface

ACI covers both the action interface and the feedback the model receives.

- **Motivation:** About when to use the tool and its limits, makes the agent's next decision and tool selection easier.
- **Action interface:** Clear operations (functions) definition and valid inputs (arguments/parameters).
- **Observation interface:** Useful and clear results, status and errors.



4 Tool granularity

How much work a single tool call covers? Does the tool have enough reusability?

Fine-grained

```
lookup_city()
find_station()
fetch_forecast()
```

- More calls and latency.
- More intermediate arguments.
- More coordination errors.

Focused operation

```
get_weather(
  city, date
)
```

- Hides routine lookup.
- Keeps saving separate.
- Retains a clear effect.

Broad workflow

```
plan_entire_trip()
```

Weather, booking, payment?

- Mixed goals and effects.
- Harder to inspect failure.
- Retries may repeat writes.

Combine routine steps. Preserve decisions that change scope or effects.

4 MCP: purpose and architecture

- **Motivation:** Reuse integrations across compatible applications.
- **Model Context Protocol:** an open protocol connecting AI applications to external tools and context.

HOST APPLICATION Model + MCP client	MCP MESSAGES	MCP SERVER Tool implementations
1 Discover server (optional for clients)	`Request: server/discover` →	
	← Response: server information	Supported versions and capabilities
2 List tools	`Request: tools/list` →	
	← Response: tool definitions	Names, descriptions and input schemas
3 Choose + check Model proposes a call. Host checks inputs + access.		
4 Call the tool	`Request: tools/call` →	
	← Response: result or error	Validate inputs + access Execute selected tool

4 Example: Discovery request and candidate result

Illustrative application catalog JSON, with summaries before full definitions.
These are customized catalog fields. Standard MCP tools/list enumerates tool definitions.

Request

```
{
  "request_id": "discover_1",
  "query": "Forecast for Beijing",
  "capability": "perception",
  "limit": 2
}
```

Next: inspect the forecast tool's full definition and supported dates.

Candidate result

```
{
  "request_id": "discover_1",
  "candidates": [
    {
      "name": "get_weather",
      "summary": "City and date forecast",
      "definition_ref":
        "weather/get_weather"
    },
    {
      "name": "get_weather_history",
      "summary":
        "Past weather observations",
      "definition_ref": "weather/history"
    }
  ]
}
```

4 Example: MCP tool definition and call

Definitions describe the input contract. A call supplies concrete arguments.

Tool object returned by tools/list

```
{
  "name": "get_weather",
  "description":
    "Forecast for a city and date.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "city": {"type": "string"},
      "date": {
        "type": "string",
        "description": "YYYY-MM-DD"
      }
    },
    "required": ["city", "date"],
    "additionalProperties": false
  }
}
```

tools/call parameters

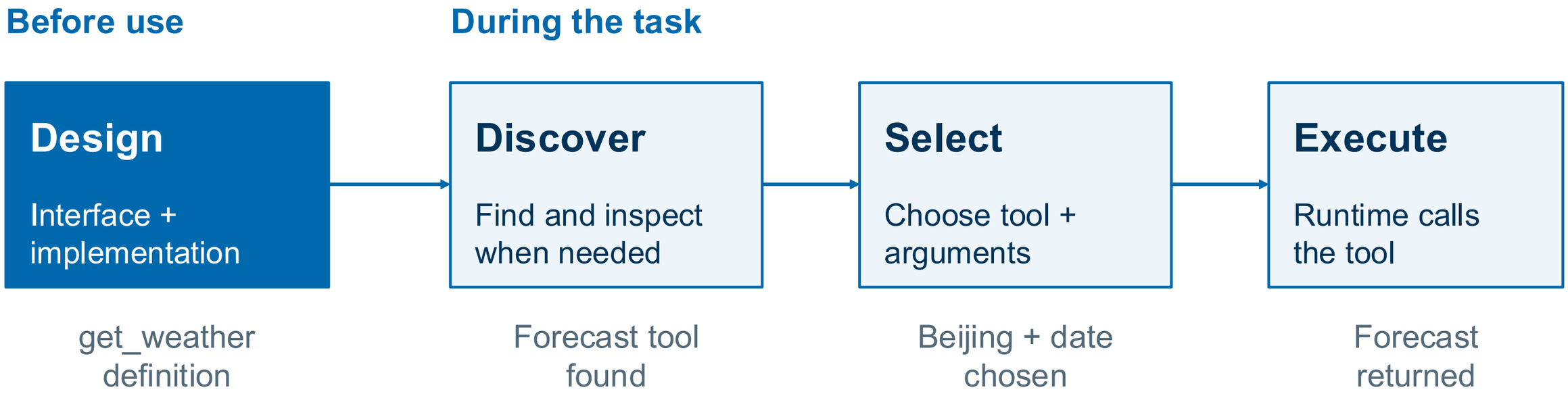
```
{
  "name": "get_weather",
  "arguments": {
    "city": "Beijing",
    "date": "2026-09-18"
  }
}
```

Method-specific fields shown here.
A full request also needs JSON-RPC and the standard_meta fields.

MCP arguments is an object. Keep it distinct from a model API’s tool-call format.

4 Tool design, discovery and selection

- **Design:** How should a tool work and describe itself?
- **Discovery:** What relevant tools can I find?
- **Selection:** Which tool should I use next?



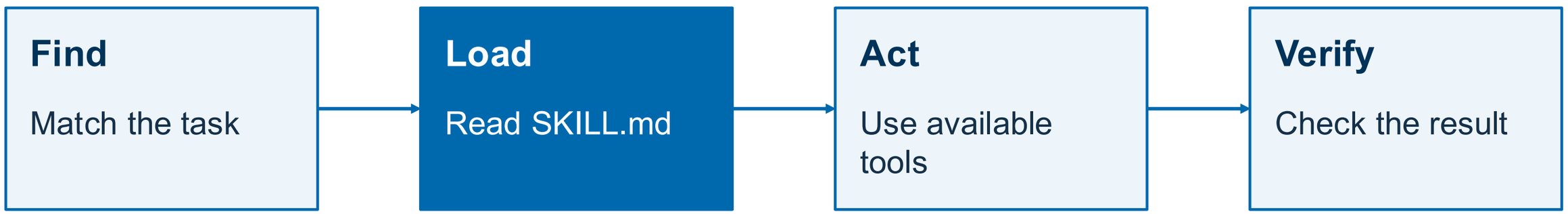
Discovering a tool does not execute it. The runtime executes the call.

5 Skills

Reusable task instructions and resources that guide an agent's workflow.

- **Motivation:** Reuse domain procedures and completion checks.
- **Tool package:** SKILL.md plus optional scripts, references and assets.
- **Activation:** Load relevant instructions into the model's context.

(SKILL) Travel Briefing: check weather, prepare advice, save when requested.



The Skill guides the procedure. Tools perform the operations.

5 Example: SKILL.md

- **Frontmatter**: when to use it.
- **Markdown body**: steps and completion checks.

travel-briefing/SKILL.md

```
---
name: travel-briefing
description: >
  Prepare weather-based travel advice.
  Use for a city-and-date travel briefing.
---

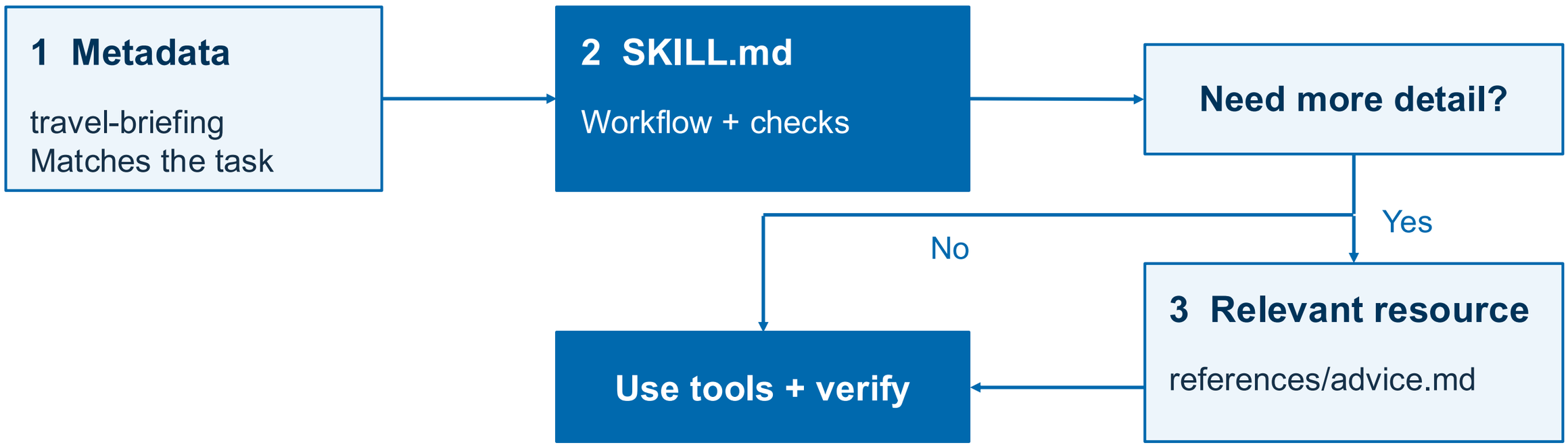
# Travel Briefing
1. Confirm the city, date and requested output.
2. Call get_weather with valid arguments.
3. Ground advice in the returned forecast.
4. If saving is requested, call save_note.
5. Check the result before reporting completion.

# Exception
If a tool fails, explain what remains incomplete.
For extra advice guidance, read references/advice.md.
```

5 Skill progressive disclosure

Load more detailed instructions only when the current task needs them.

- **Metadata:** Name + description help choose a Skill.
- **Instructions:** The selected SKILL.md provides the workflow.
- **Resources:** Read references or use supporting files as needed.



Less irrelevant context upfront. Loaded content still occupies context.

5 Skill Hub and MCP

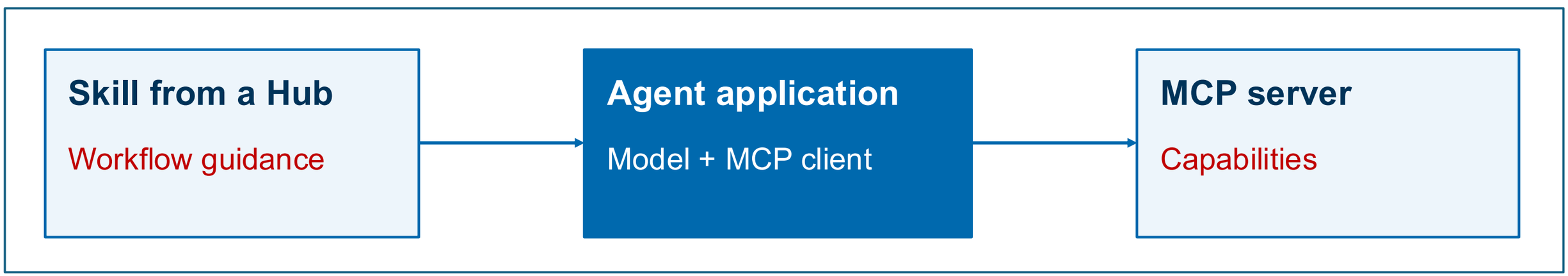
- Skill Hub: A catalog concept for reusable workflows (Skills).
- MCP: A protocol for capability access (tools).
- Skill: Guide the use of MCP tools.

Skill Hub

- **Role:** Find and distribute Skill packages.
- **Unit:** Instructions with supporting files.
- **Example:** Locate travel-briefing.

MCP

- **Role:** Standardize client–server communication.
- **Unit:** Tool, resource and prompt interfaces.
- **Example:** Discover and call get_weather.

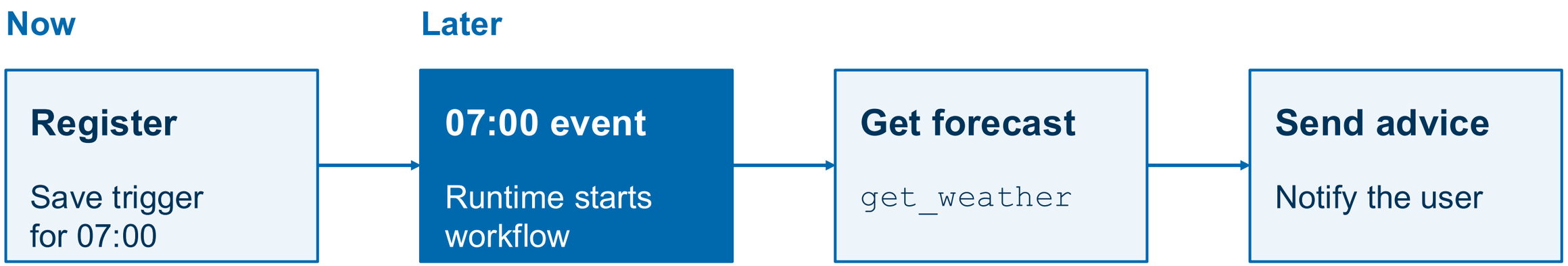


5 Event-triggering

Signals that alert something has happened, used by the runtime to start or resume processing.

- A scheduled time arrives.
- A background task finishes.
- A new message or service callback arrives.

Example: 07:00 weather briefing



Registration sets the trigger. The later event starts processing.
Further actions complete the task.

5 Plugins – Example: “Computer Use”

Installable extension packages that add capabilities to an agent application.

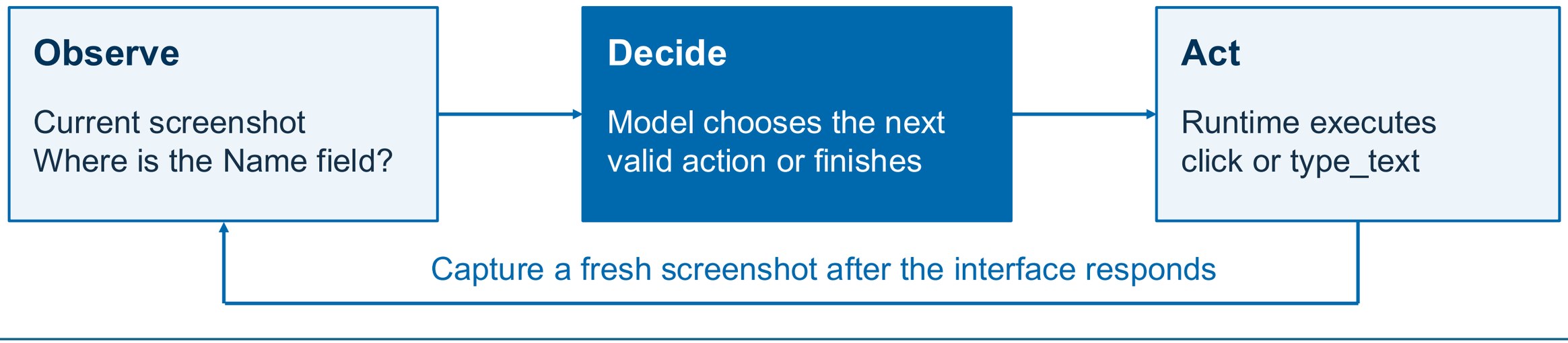
Observation space

- Screenshots of visible interface states.
- Cursor position or status, if exposed.

Action space

- screenshot()
- click(x, y), double_click(x,y)
- type_text(text), key(key_name)
- scroll(direction)

Example: Fill the Name field with “Ada”.
Check: “Ada” must appear in Name, not in another field.



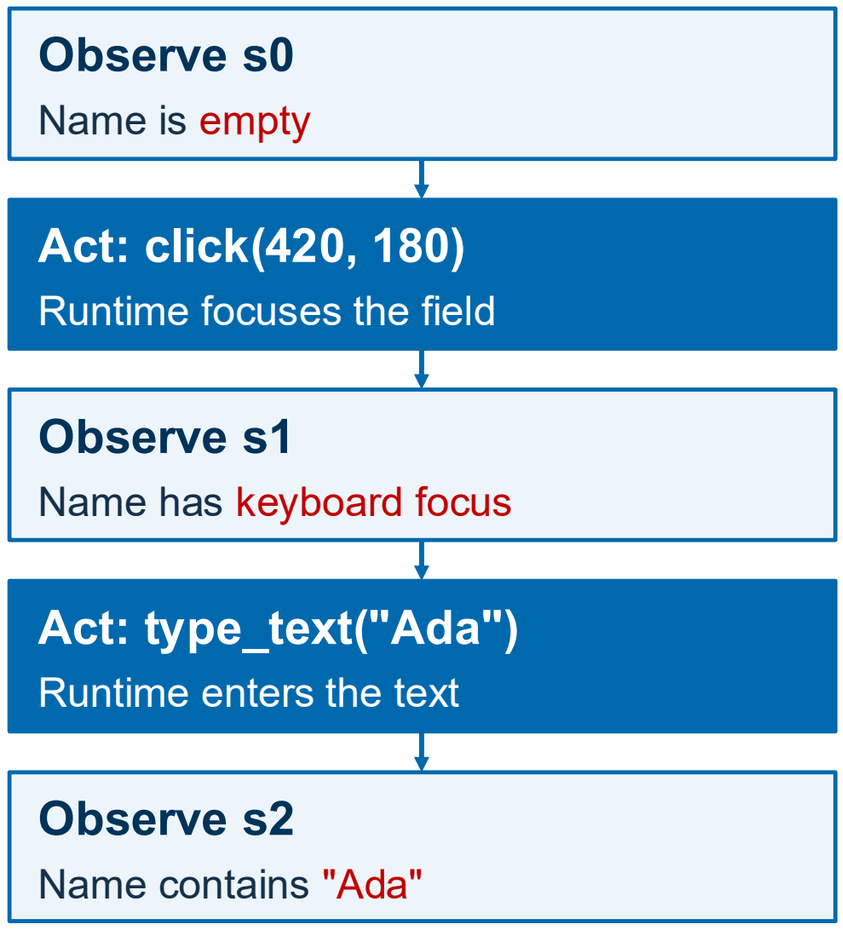
5 Example: “Computer Use” fill the Name field

User request: set Name to "Ada" in a practice form, without submitting it.

Illustrative JSON trace

```
[
  {"action": "screenshot",
   "observation": {"image_ref": "s0"}},
  {"action": "click",
   "args": {"x": 420, "y": 180},
   "observation": {"status": "executed"}},
  {"action": "screenshot",
   "observation": {"image_ref": "s1"}},
  {"action": "type_text",
   "args": {"text": "Ada"},
   "observation": {"status": "executed"}},
  {"action": "screenshot",
   "observation": {"image_ref": "s2"}}
]
```

Observe, act and verify



Verify, then report completion

5 Skills, events, plugins and tools

- **Skill:** How should the task be performed? (Task execution procedure)
- **Event:** What happened that triggers processing?
- **Plugin:** How are capabilities packaged and delivered? (Observation and action space extension)
- **Tool:** Which operation can the runtime execute?

